4FULFILLMENT — TECHNICAL DOCUMENTATION

# CMC CartonWrap -- Process Documentation

Process flow, state lifecycle, error handling, and manual fallback — March 2026

> **Version 2.0** | 2026-03-10 | [Changelog](#changelog)

This document describes the end-to-end flow of orders through the CMC CartonWrap machine, from the initial barcode scan to label generation and completion. It covers the physical machine protocol, software state management, error handling, manual fallback, and how the system prevents duplicate or lost shipments.

---

## Table of Contents


1. [System Overview](#1-system-overview)
2. [Physical Machine Flow](#2-physical-machine-flow)
3. [Barcode Scanning and Order Routing](#3-barcode-scanning-and-order-routing)
4. [State Lifecycle](#4-state-lifecycle)
5. [Label Generation](#5-label-generation)
6. [Label Types and Pre-Creation](#6-label-types-and-pre-creation)
7. [Error Handling and Recovery](#7-error-handling-and-recovery)
8. [Manual Fallback -- Packing Without the Machine](#8-manual-fallback----packing-without-the-machine)
9. [Manual Resolution of Machine Issues](#9-manual-resolution-of-machine-issues)
10. [End-to-End Scenarios](#10-end-to-end-scenarios)

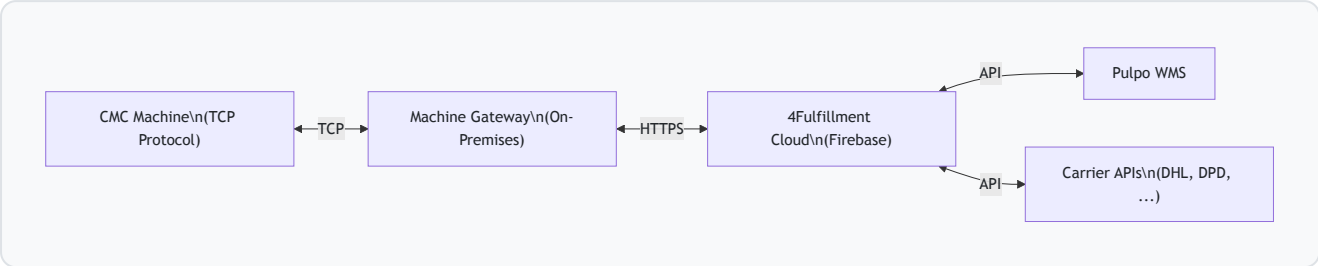

---

## 1. System Overview

The CMC CartonWrap is an automated packing machine that wraps products in custom-sized cardboard, applies shipping labels, and verifies the result -- all on a continuous conveyor belt. Multiple packages travel through the machine simultaneously at different stages.

Three systems work together:

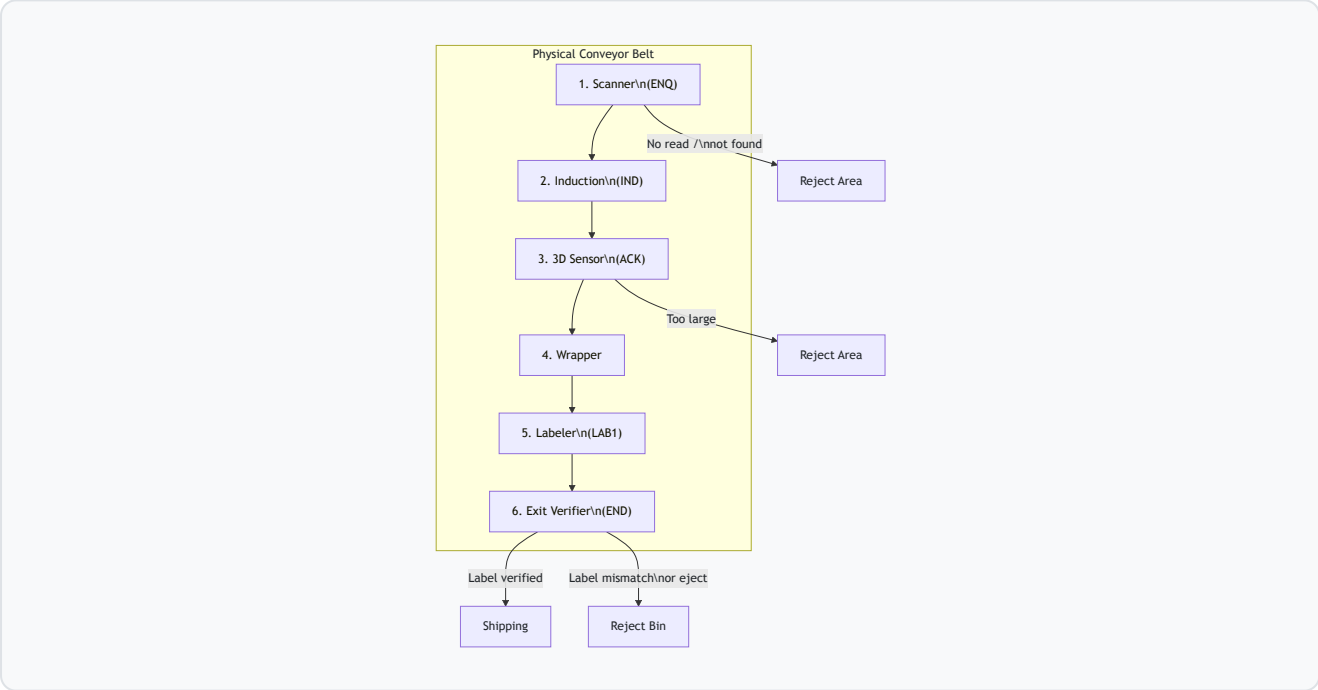
- **CMC Machine** -- the physical hardware (conveyor, scanner, 3D sensor, wrapper, labeler, verifier)
- **Machine Gateway** -- on-premises software that translates between the machine's TCP protocol and the cloud
- **4Fulfillment Cloud** -- manages order assignment, label generation, and warehouse completion




---

## 2. Physical Machine Flow

A package moves through six stations on the conveyor belt. At each station, the machine exchanges a protocol message with the cloud.



| Station | Protocol Event | What Happens |
|---|---|---|
| **Scanner** | ENQ (Enquiry) | Barcode is read. Cloud looks up the order and decides accept/reject. |
| **Induction** | IND | Item enters the conveyor. Cloud fetches order items from the Firebase packing queue cache (falls back to Pulpo API on cache miss). If pre-creation is enabled, triggers async label generation. |
| **3D Sensor** | ACK (Acknowledge) | Dimensions are measured. Cloud stores them. If out of spec, the item is ejected. |
| **Wrapper** | *(no message)* | Machine selects and folds cardboard, wraps the product, closes the box. |
| **Labeler** | LAB1 | Machine weighs the box and requests a shipping label. If a pre-created label exists (from IND), cloud reuses it immediately. Otherwise, cloud calls the carrier API and returns the label. |
| **Exit Verifier** | END | Machine scans the label barcode and compares it to the expected value. Reports success or failure. |

Additional events:

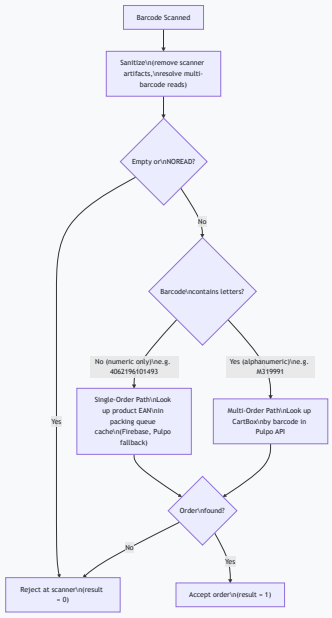
- **REM** (Remove) -- Operator manually removes an item mid-process (jam, wrong product). Cloud cleans up the machine state; the order remains in the Pulpo queue (it was never accepted) and can be re-processed.
- **HBT** (Heartbeat) -- Periodic connection check.

---

## 3. Barcode Scanning and Order Routing

When the scanner reads a barcode, the system must determine which order it belongs to. There are two routing paths, selected automatically based on the barcode format.

**Multi-barcode scanner input:** Some scanners return multiple barcodes in a single read, separated by semicolons (e.g. `4052400033054;M0000298`). The sanitizer splits these and applies priority selection: M-prefixed barcodes first, then other alphanumeric, then numeric. This means if both an EAN and a cart box label are read simultaneously, the M-prefixed barcode wins and routes to the Multi-Order Path. If only numeric EANs are present (e.g. `4052400033054;8711319002345`), the first EAN is selected and routes to the Single-Order Path.




### Packing Queue Cache

The single-order path relies on a high-performance Firebase packing queue cache instead of querying Pulpo on every scan. This cache is kept in sync via two Pulpo webhooks:

- `packing_order_created` -- adds the order to the cache (barcode, items, shipping method, location)
- `packing_order_finished` -- removes the order from the cache

If the cache lookup throws an error and the cache is fresh, the ENQ handler falls back to Pulpo API for that single request. If a barcode scan results in a cache miss (no match) and the cache is outdated (freshness check fails), the ENQ handler triggers a full resync from Pulpo (`PackingQueueSyncService.sync()`) as self-healing. The current scan still returns "not found", but the cache is now up to date for the next scan.

### Single-Order Path (Numeric EAN/GTIN Barcodes)

Used when the operator scans a product's EAN barcode directly.

1. Query the Firebase packing queue cache for orders in "queue" state at the machine's configured pick locations (falls back to Pulpo API on cache error; auto-resyncs stale cache on miss)
2. Check which orders are already reserved by an active machine state (to avoid double-assignment)
3. Match the scanned barcode against product barcodes in each order's items
4. If multiple orders match, select the oldest (FIFO) to ensure pick carts drain efficiently

### Multi-Order Path (Alphanumeric Barcodes)

Used when the operator scans a cart box label (e.g. `M319991`) that was applied during picking.

1. Look up the CartBox in Pulpo by barcode
2. Extract the linked fulfillment or sales order

3. Find the corresponding packing order in "queue" state
4. Verify the order is at the machine's configured location

After routing, both paths create the same machine state and follow the same conveyor flow.

---

## 4. State Lifecycle

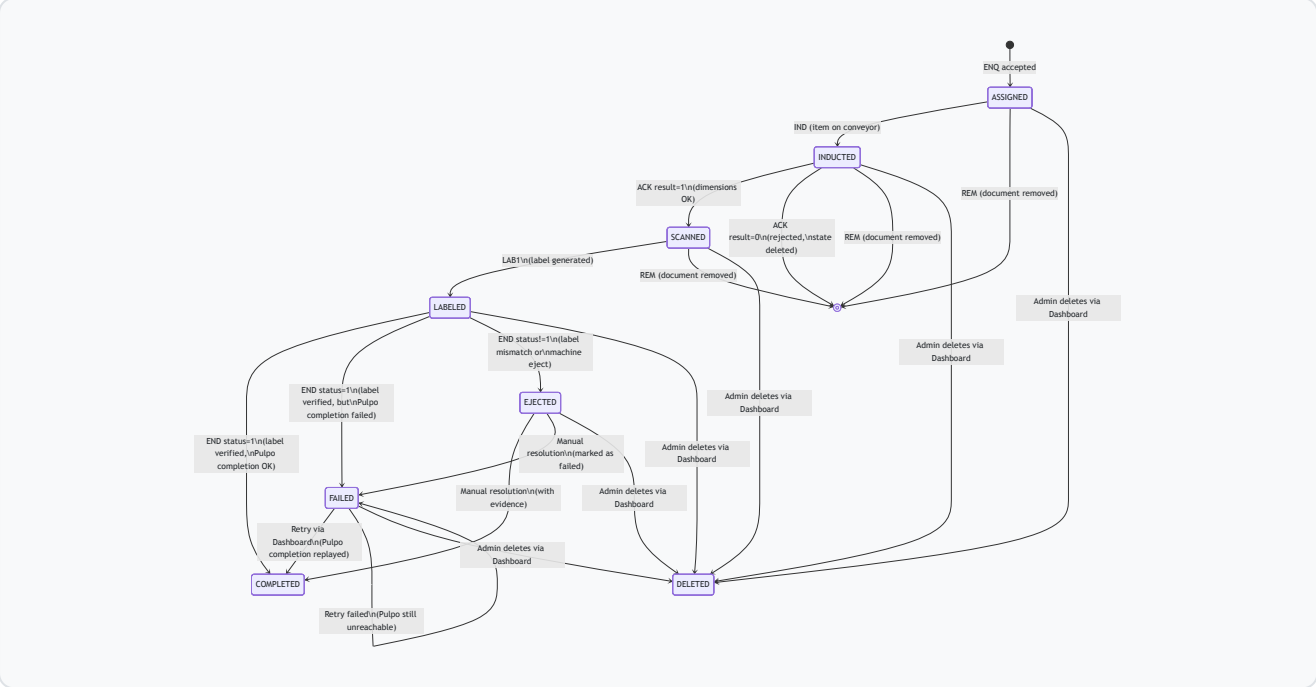
Every order passing through the machine is tracked by a **CMC State** document. This state moves through well-defined transitions as the package progresses along the conveyor.

### State Categories

| Category | States | Meaning |
|---|---|---|
| **Active** | ASSIGNED, INDUCTED, SCANNED, LABELED | Package is physically on the conveyor. Order is protected from duplicate processing. |
| **Terminal** | COMPLETED, FAILED, EJECTED, DELETED | Package has left the conveyor or requires intervention. Packing space is freed. Order can be processed manually. |

Note: **FAILED** is terminal (the package has left the conveyor), but it also **blocks re-scanning** on the single-order path. A FAILED state means the package was labeled and exited the machine successfully, but the subsequent END step failed (Pulpo API error, handler exception, or other failure). Since the Pulpo completion failed, the packing order still exists in Pulpo's queue -- it was never accepted, finished, or closed. Without the guard, this order would be picked up again by the next matching barcode scan, triggering a new ENQ, potentially pre-creating a new label at IND, generating a new shipment at LAB1, and producing a duplicate tracking number -- while the original labeled package is already on its way to the customer. By including FAILED in the reservation guard (same as active states), re-scanning is rejected at ENQ (`result=0`), preventing duplicate parcels. The order must be resolved via Dashboard retry, which replays the Pulpo completion using the existing label and tracking number.

# State Transition Diagram



### What Each State Means

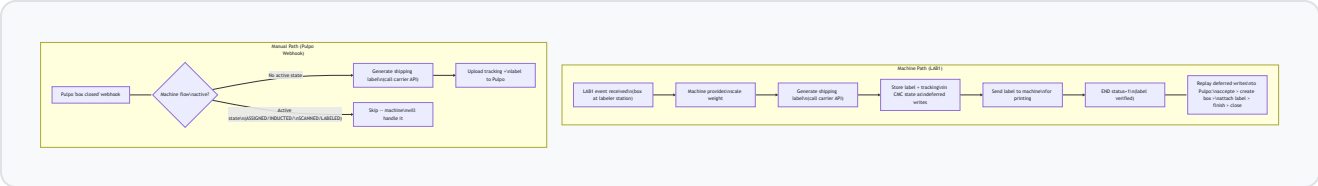
| State | Trigger | What happened |
|---|---|---|
| **ASSIGNED** | ENQ accepted | Order matched to barcode, reserved for machine processing. Awaiting physical induction. |
| **INDUCTED** | IND received | Item is on the conveyor. Order items fetched from Pulpo. |
| **SCANNED** | ACK with result=1 | 3D scan complete, dimensions within spec. Ready for labeling. |
| **LABELED** | LAB1 processed | Shipping label generated and sent to machine for printing. Contains tracking number and deferred Pulpo writes. |
| **COMPLETED** | END with status=1 | Package exited successfully. Label verified. Pulpo order finished and closed. Auto-cleaned after 48 hours. |
| **FAILED** | END success but Pulpo error | Machine succeeded, but the Pulpo completion calls failed. Package has a valid label but WMS is out of sync. Requires manual resolution. |
| **EJECTED** | END with status!=1 | Machine ejected the package (label mismatch, timeout, mechanical issue). No Pulpo writes were executed. Can be retried or resolved manually. |
| **DELETED** | Manual deletion via Dashboard | Soft-delete performed by an admin through the CMC Dashboard. Can be applied to any state. Used when a state blocks processing and cannot be resolved via retry — for example, a FAILED order where retry is not possible, or an active state (ASSIGNED, INDUCTED, etc.) where the parcel was physically removed from the machine. Sets state to DELETED and records `previous_state_before_delete`, `deleted_by`, and `deleted_at` for traceability. Treated as terminal, so the packing order is unblocked for re-processing. Auto-cleaned after 48 hours. |

> **Warning:** Unlike FAILED, a DELETED state does **not** block re-scanning. Once a state is deleted, the packing order becomes available again on the single-order path — the next matching barcode scan will trigger a new ENQ and start a full new processing cycle. If the original package was already labeled (FAILED or LABELED state before deletion), this will create a duplicate shipment. Admins must ensure the original parcel is intercepted or the shipment is voided **before** deleting the state, to avoid duplicate parcels shipping to the customer.

---

## 5. Label Generation

Label generation happens in **two distinct paths** depending on whether the order is processed by the machine or packed manually.




### Why Deferred Writes?

When the machine processes a package, label generation happens at the LAB1 station, but Pulpo completion must wait until the END event confirms the package exited successfully. Between LAB1 and END, all Pulpo state changes are stored as "deferred writes." Only after the machine confirms success at END are these writes replayed to Pulpo in sequence:

1. Accept the packing order
2. Create/update the box with dimensions and weight
3. Attach the shipping label and tracking number
4. Finish the packing order
5. Close the packing order

This prevents partial state in Pulpo if the machine ejects the package after labeling.

---

## 6. Label Types and Pre-Creation

The machine supports multiple label sources and an optional pre-creation flow that generates labels earlier in the process to minimize waiting time at the labeler station.

### Label Sources

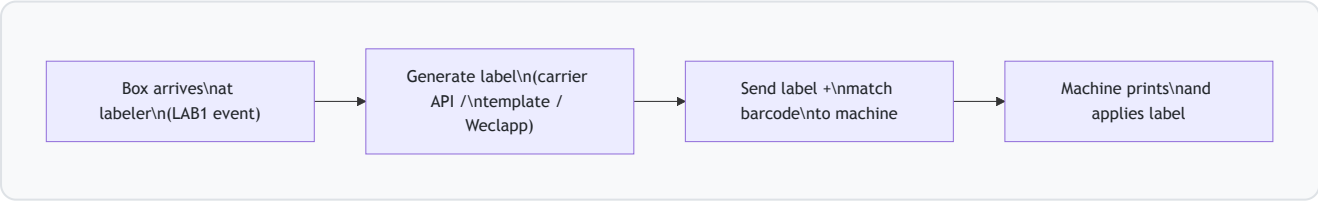
Depending on the tenant's shipment method configuration, the machine generates one of three types of labels:

| Label Source | How It Works | Match Barcode (Exit Verification) | Tracking Number |
|---|---|---|---|
| **Carrier label** (DHL, DPD, FedEx, ...) | Calls the carrier's API to create a shipment and retrieve a shipping label PDF with tracking number. | Carrier tracking number | Yes -- provided by carrier |
| **Template label** | Renders a custom PDF from a tenant-configured HTML template. Used for internal logistics, self-delivery, or cases where no carrier API is needed. | Order number (from Pulpo/Weclapp) | None -- no external shipment created |
| **Weclapp document label** | Downloads a pre-existing document from Weclapp (e.g. a delivery note or custom label) and uses it as the machine label. Configured via `weclappDocumentPrinting` with `replacementMode: replace`. | Order number (from Weclapp) | None -- document already exists in Weclapp |

All three label types follow the same machine flow: the label is sent to the machine for printing, and the exit scanner verifies the match barcode regardless of the label source.

### Standard Labeling Flow (at LAB1)

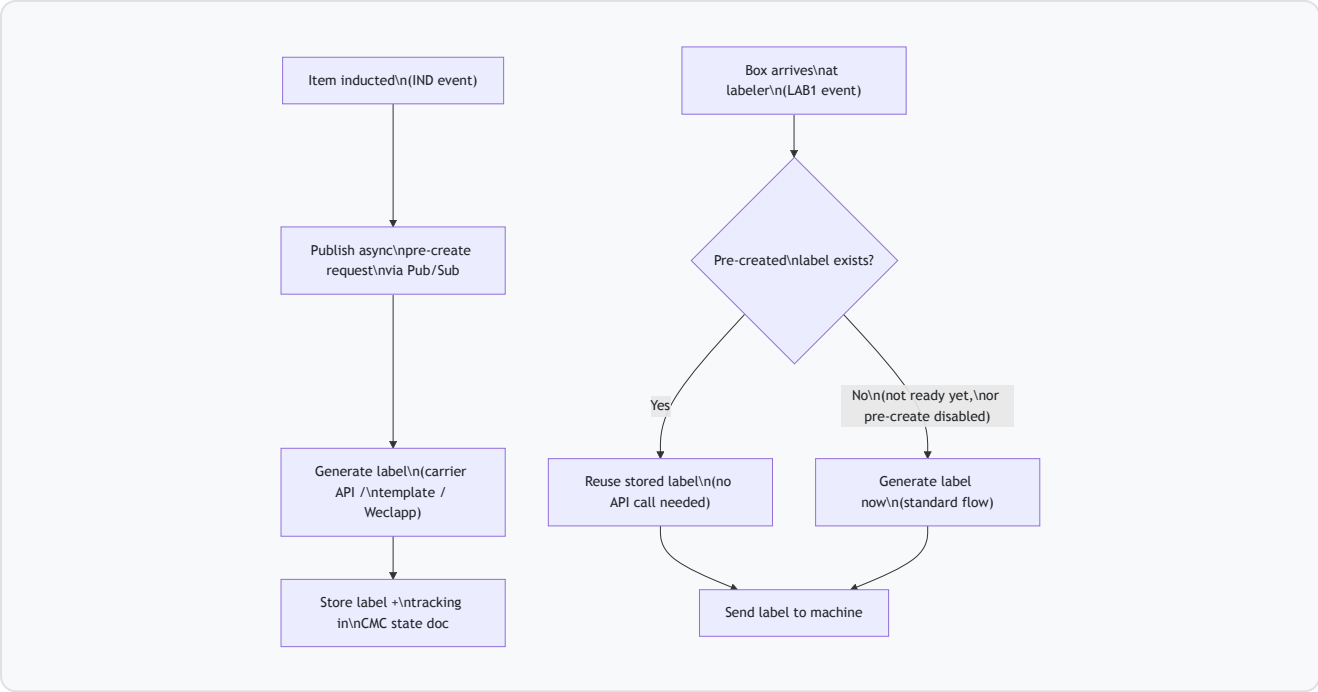
In the default flow, label generation happens at the LAB1 station -- when the box arrives at the labeler. The machine sends a LAB1 event, the cloud generates the label, and responds with the label file and match barcode.




This is simple and reliable, but the label generation call (especially carrier API calls) adds latency. The machine must wait for the response before printing.

### Pre-Creation Flow (at IND)

To reduce waiting time at the labeler, label generation can optionally be triggered earlier -- at the IND (induction) station, when the item first enters the conveyor. The label is generated asynchronously via Pub/Sub while the item continues moving through the 3D sensor and wrapper.




Pre-creation is:

- **Enabled by default** at the machine level (`pre_create_labels: true` in machine config)
- **Overridable per carrier** via `cmcPreCreateLabelsOverride` in the shipment method's carrier config (e.g. disable pre-creation for carriers with strict weight requirements, since the exact machine weight is only available at LAB1)
- **Gracefully degraded** -- if the pre-created label is not ready by the time LAB1 fires, the system falls back to standard generation. No error, no delay beyond the normal label generation time.

### Orphan Label Handling

Pre-creation introduces a risk: a label may be generated for an order that is later rejected by the machine (e.g. the 3D sensor finds the item too large at the ACK station). If this happens, a shipping label and tracking number exist but the package was never shipped.

The system handles this by deleting the state document -- the orphan label is silently discarded, and the order returns to the Pulpo queue for re-processing. When the order is re-scanned (on the machine or at a manual station), a new label is generated.

| Scenario | Pre-Created Label? | System Behavior |
|---|---|---|
| Item accepted by 3D sensor | Yes or No | Normal flow continues |
| Item rejected | No | State deleted, order returns to Pulpo queue. Clean. |
| Item rejected | Yes | State deleted, orphan label discarded. Order returns to Pulpo queue for re-processing. |

### When to Disable Pre-Creation

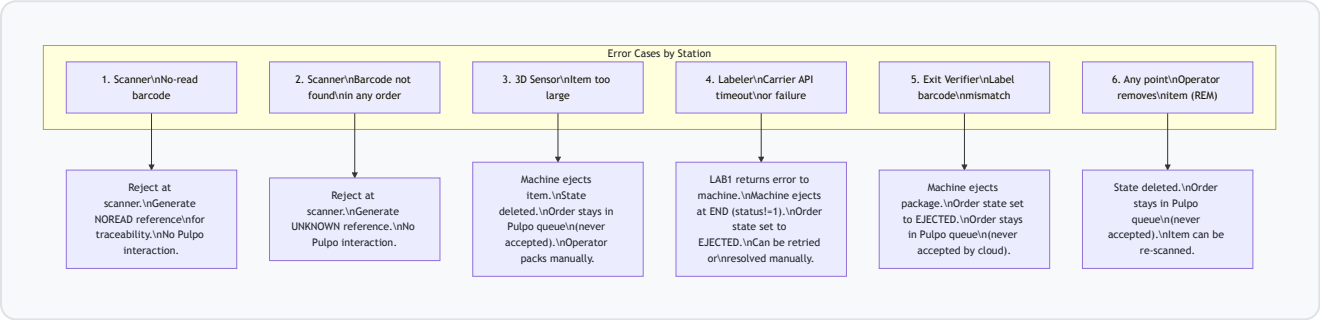
Pre-creation should be disabled when:

- The carrier requires **exact machine-weighed weight** for label generation (pre-creation uses estimated weight from the order, the actual scale weight is only available at LAB1)
- The carrier has **strict rate-limiting** on label creation (pre-creation adds an extra API call if the item is later rejected)
- **Orphan label cost** is a concern (some carriers charge per label regardless of whether the package is shipped)

---

## 7. Error Handling and Recovery

Errors can occur at five critical points along the conveyor. The system handles each differently.



### Error Details

| # | Error | Station | Physical Result | System Recovery |
|---|---|---|---|---|
| 1 | **No-read barcode** | Scanner | Item diverted to reject area | NOREAD reference generated. No API calls made (short-circuit). |
| 2 | **Unknown barcode** | Scanner | Item diverted to reject area | UNKNOWN reference generated. Logged for investigation. |
| 3 | **Item too large** | 3D Sensor | Item ejected or machine stops (if extremely oversized) | State deleted. No Pulpo API calls needed -- the order was never accepted in Pulpo due to the deferred write architecture, so it remains in queue automatically. Operator packs manually at a regular packing station. |
| 4 | **Label generation failed** | Labeler | LAB1 returns error response; machine ejects at exit | Machine receives the error at LAB1 and flags the item as "bad." At END (status!=1), state is set to EJECTED. Can be manually resolved or item re-scanned. |
| 5 | **Label verification mismatch** | Exit Verifier | Package ejected to reject bin | State set to EJECTED. No Pulpo API calls were made (deferred writes are only replayed on END status=1), so the order remains in Pulpo queue automatically. Item can be re-introduced to the machine immediately. |
| 6 | **Operator removes item** | Any | Operator physically extracts item | State deleted. No Pulpo API calls needed -- order stays in queue (never accepted). Item can be re-scanned or packed manually. |

### Exit Scanner Verification

The exit scanner is a mechanical QA check. It compares the barcode on the physical label against the tracking number value sent in the LAB1 response. This catches:

- Wrong label printed
- Smeared or damaged print
- Label applied in wrong orientation
- Printer malfunction

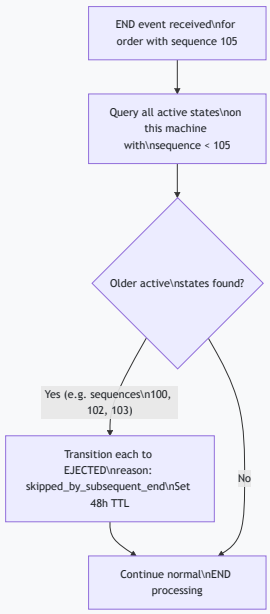
The machine makes the pass/eject decision *before* sending the END message. The END message informs the cloud what happened -- it is not asking for a decision.

### Automatic Recovery Mechanisms

The system has three built-in mechanisms that prevent orders from getting lost or stuck without manual intervention.

### 1. Sequence-Based Ejection at END

Every order entering the machine receives a monotonically increasing sequence number (`enq_sequence`), stored in Firestore and never reset. When an order reaches the END station, the system automatically ejects all older active states on the same machine:




This handles cases where an order was physically removed from the conveyor without a REM event (e.g. during a machine stoppage or jam). If a later order successfully exits, any earlier order that never reached END must have been removed -- the system ejects those states automatically.

### 2. Order Reservation at ENQ

When a barcode is scanned at the ENQ station, the system queries all CMC states with state in ASSIGNED, INDUCTED, SCANNED, LABELED, or FAILED to build a set of already-reserved `packing_order_id` values. Orders with a matching state are excluded from matching, preventing the same order from being processed by the machine twice.

This means:

- If an order is already ASSIGNED/INDUCTED/SCANNED/LABELED/FAILED, re-scanning the same barcode will not create a duplicate state
- FAILED is included because the package was already labeled — re-processing would create a duplicate shipment (see Section 4 note on FAILED). Note: the FAILED guard applies to the single-order path only. The multi-order (CartBox) path does not include FAILED in its reservation query, but this is acceptable because multi-order barcodes uniquely identify a single packing order — re-scanning the same CartBox barcode would resolve to the same failed order, not a different one
- The barcode is rejected at the scanner, and the item goes to the reject area
- The operator/admin must resolve the existing state first via the Dashboard (or let the sequence-based ejection handle active states)

### 3. REM (Remove) Event

When an operator physically removes an item from the conveyor while the machine is running, the machine sends a REM event. The system:

- Deletes the machine state
- The order remains in the Pulpo queue automatically (it was never accepted due to deferred writes)
- The item can be re-scanned immediately or packed manually

---

## 8. Manual Fallback -- Packing Without the Machine

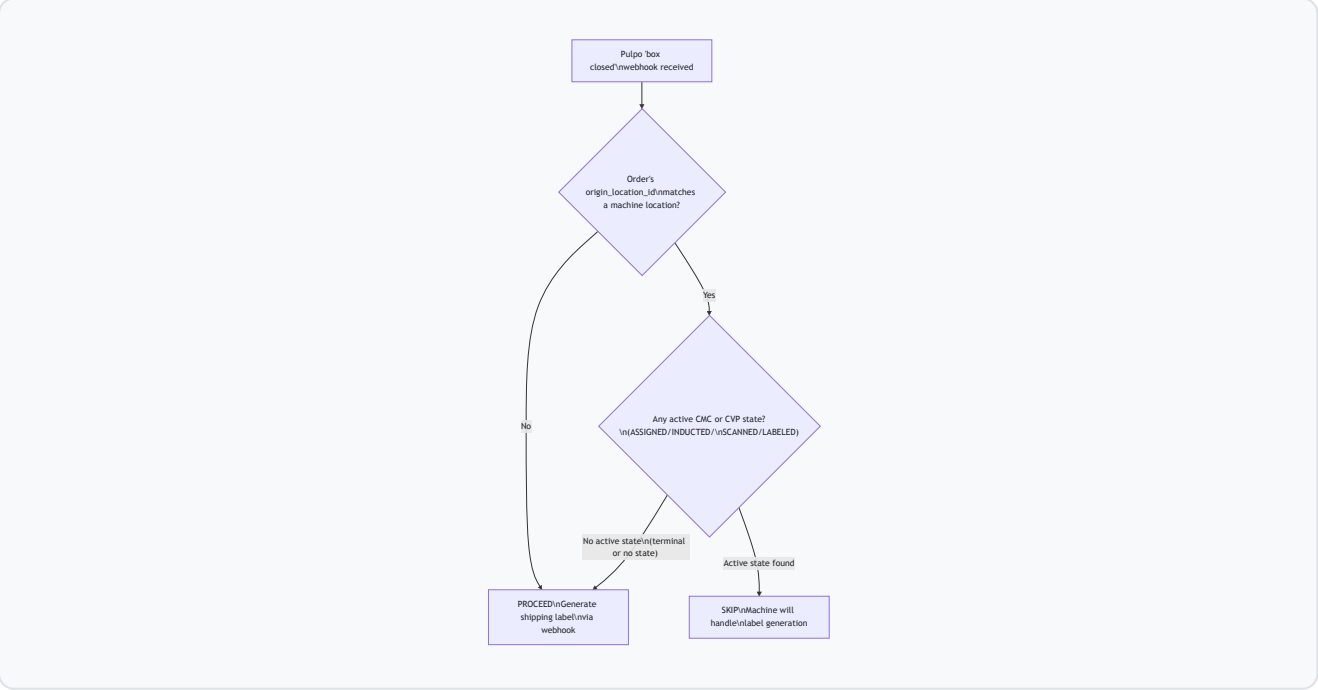
Orders picked to a machine location can sometimes not be processed by the machine (too large, machine failure, maintenance). These orders must still be packable at a regular packing station.

### The Problem

When an operator packs an order manually at a regular packing station, Pulpo sends a "box closed" webhook to generate the shipping label. Without safeguards, an order simultaneously on the machine conveyor could receive two labels -- one from the machine and one from the webhook.

### The Solution: State-Aware Skip Logic

The webhook checks whether the order is **actively** being processed by a machine before generating a label.



### When Manual Fallback Is Allowed

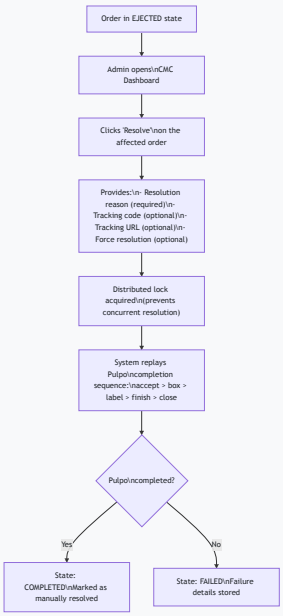
| Scenario | Machine State | Webhook Behavior |
|---|---|---|
| Order on conveyor, actively processing | ASSIGNED / INDUCTED / SCANNED / LABELED | **Skip** -- machine handles the label |
| Machine successfully completed the order | COMPLETED | **Proceed** -- safe, a downstream idempotency check detects that a tracking code already exists on the box in Pulpo (written by deferred writes at END) and skips label creation |
| Machine ejected the package | EJECTED | **Proceed** -- machine did not complete, no Pulpo writes were executed, manual label needed |
| Machine labeled but Pulpo completion failed | FAILED | **Proceed** -- not an active state. In practice this scenario is unlikely: since Pulpo completion failed (deferred writes never ran), the box was never closed in Pulpo, so Pulpo would not fire a box_closed webhook for this order |
| Order soft-deleted by admin | DELETED | **Proceed** -- state was cleared, order is unblocked for re-processing |
| Order never entered the machine | No state exists | **Proceed** -- standard manual packing |
| Machine is down, 700 orders on machine location | A few orders may have active states (those physically on the conveyor); rest have no state | **Proceed** for all orders not on the conveyor. Those on the belt are unblocked after restart ([Section 10](#machine-outage-and-recovery)) or Dashboard cleanup. |

This design ensures that during a machine failure, operators can immediately switch to manual packing for the vast majority of orders. Only orders physically on the conveyor at the time of failure remain blocked until the machine restarts or an admin resolves them via the Dashboard. See [Section 10: Machine Outage and Recovery](#machine-outage-and-recovery) for details.

---

# 9. Manual Resolution of Machine Issues

When orders end up in EJECTED state, an administrator can manually resolve them through the CMC Dashboard.




## Resolution Evidence

Every manual resolution records:

- **Who** resolved it (user ID)
- **When** it was resolved (timestamp)
- **Why** it was resolved (reason text, required)
- **Evidence** (tracking code, tracking URL, label data if available)

The CMC Dashboard displays a visual indicator on manually resolved orders so operators can quickly distinguish machine-completed from manually-completed shipments.

### When to Use Manual Resolution

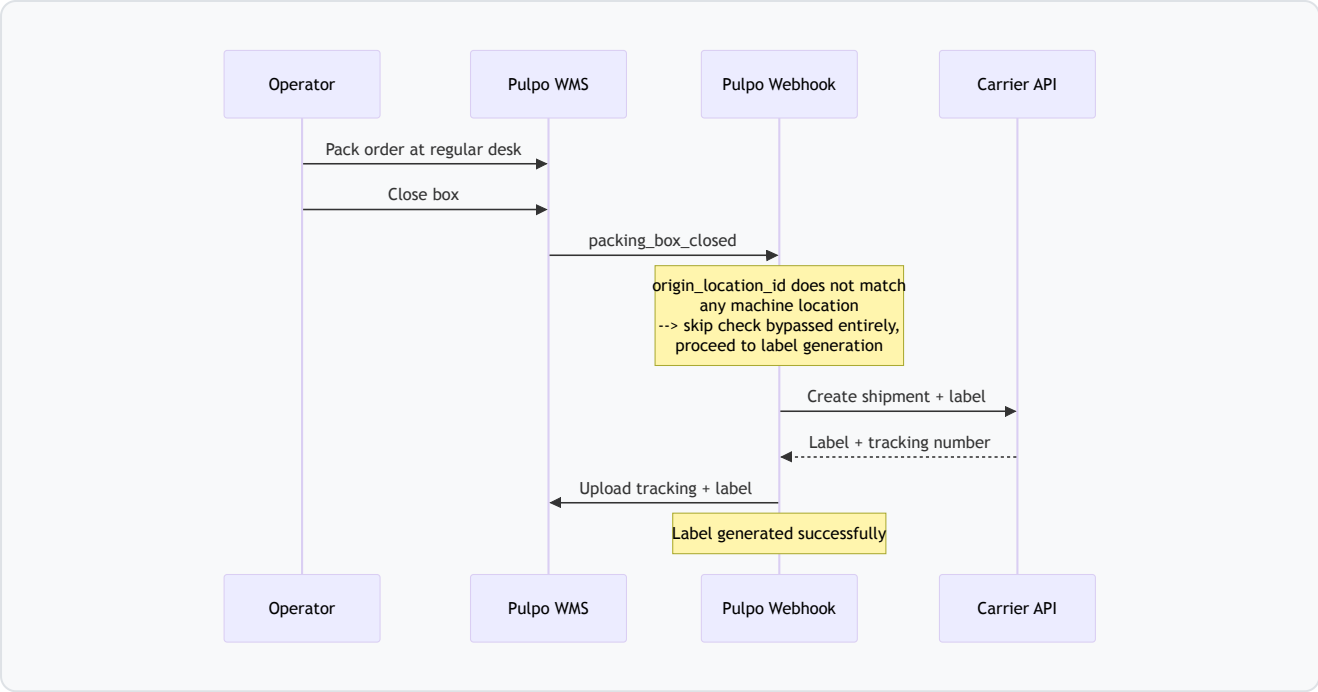
| Situation | Action |
|---|---|
| Machine ejected a package but the label was already printed and applied correctly | Resolve as completed -- package can ship as-is |
| Machine ejected a package with a damaged label | Pack and label manually at a regular station, then resolve as completed with the new tracking number |
| Pulpo completion failed after machine success (FAILED state) | Retry via resolve. System will re-attempt the Pulpo completion sequence. |
| | |

## 10. End-to-End Scenarios

This section traces three representative scenarios through the complete system -- from order picking to shipped package -- showing exactly how the webhook skip logic, machine flow, and retry mechanisms interact.

### Scenario A: Fresh Order (No Machine Involvement)

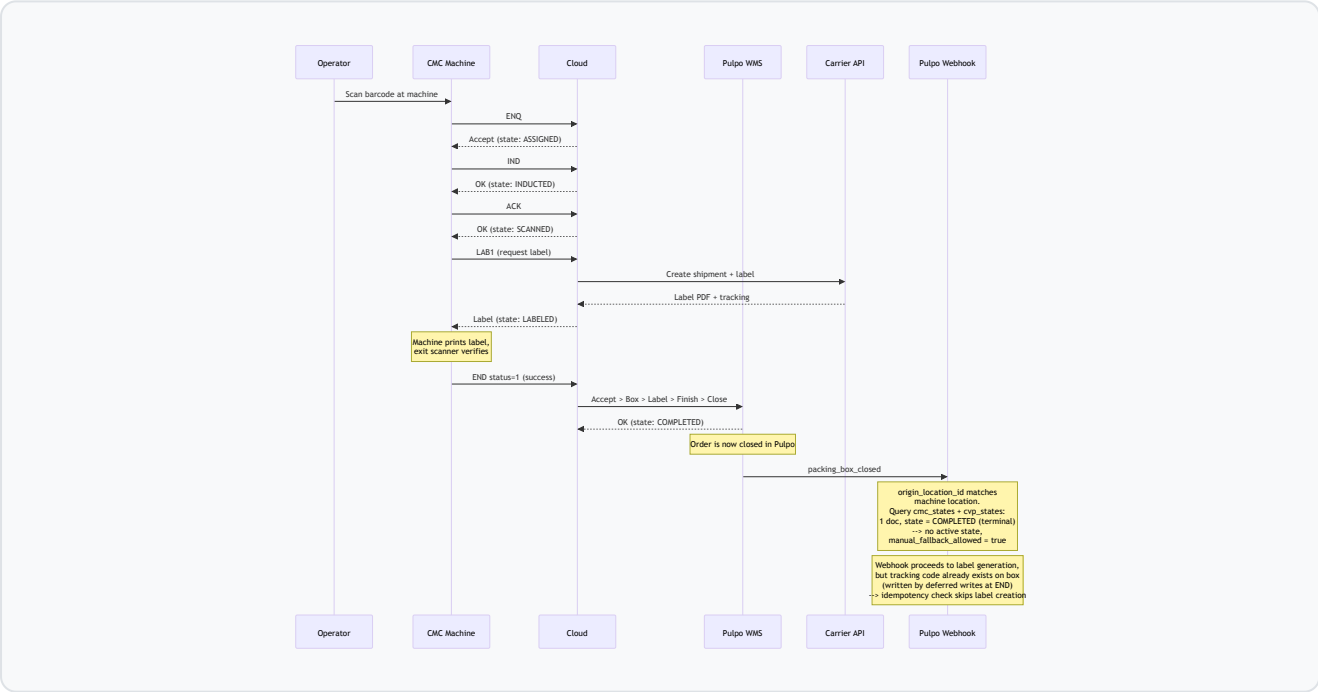
An order is picked to a regular packing location, packed at a standard desk, and closed in Pulpo.




No machine states exist, no checks block the webhook, label is generated normally.

### Scenario B: CMC Order -- Successful Machine Processing

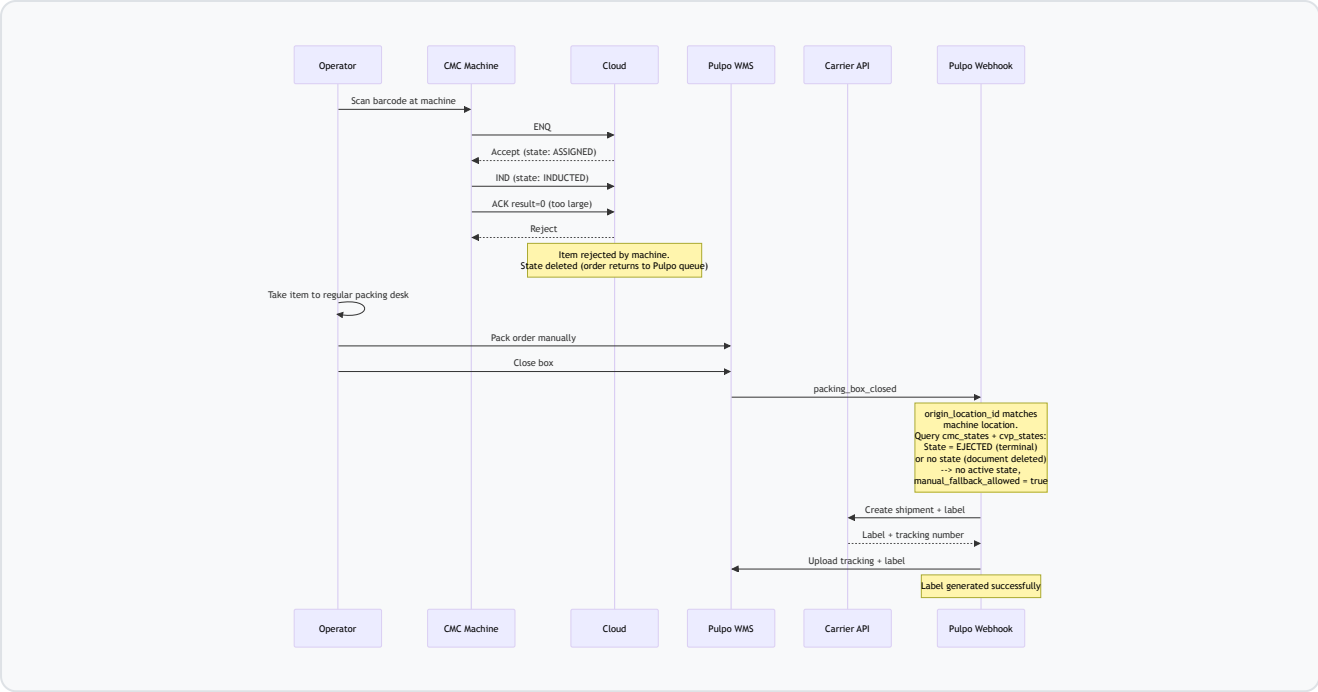
An order is picked to a CW (CartonWrap) location and processed entirely by the machine.




The machine handles everything end-to-end. The webhook fires after machine completion but the downstream idempotency check detects that a tracking code already exists on the box in Pulpo and skips label creation -- no duplicate label.

### Scenario C: CMC Order Failed -- Manual Completion via Pulpo

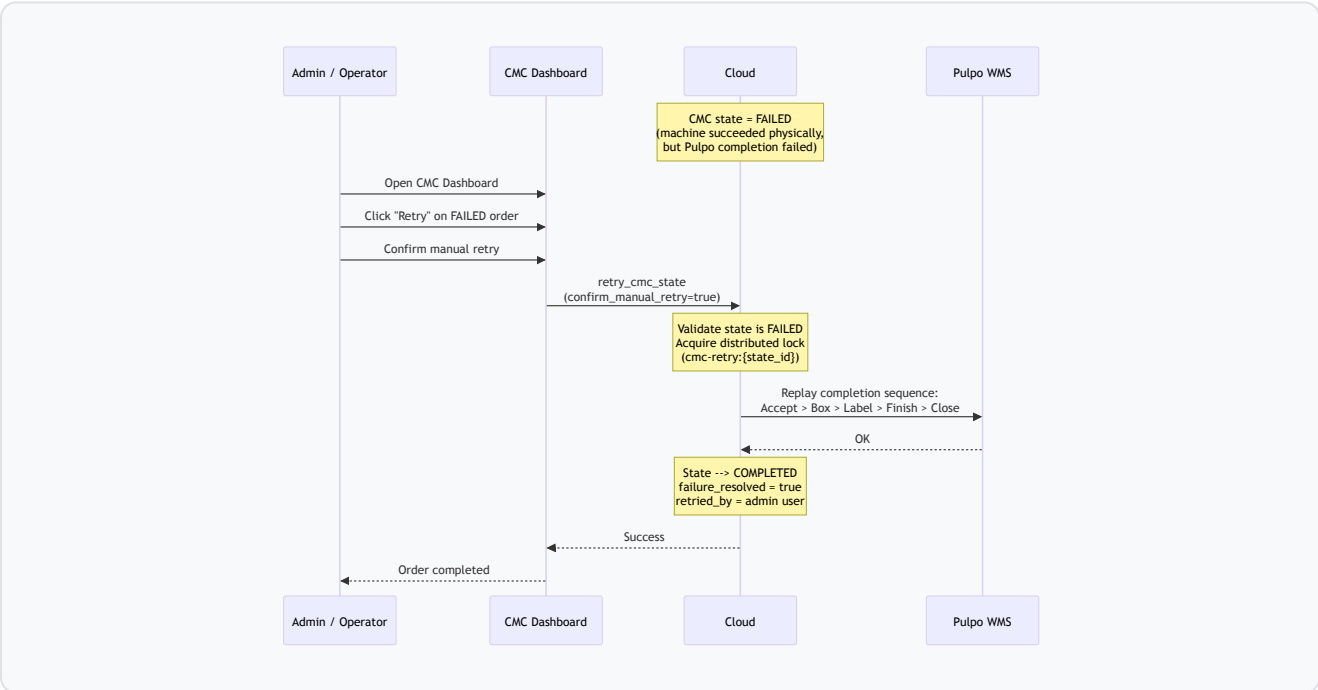
An order entered the machine but failed (e.g. too large, ejected, or Pulpo completion error). The operator repacks manually and closes the box in Pulpo.




The machine state is terminal (or absent), so the webhook proceeds normally and generates a fresh label. This is the core manual fallback: **any order not actively on the conveyor can be packed and labeled through the normal Pulpo workflow.**

### Scenario C2: CMC Order Failed -- Retry via Dashboard

Instead of repacking manually, the operator uses the CMC Dashboard to retry the Pulpo completion for a FAILED state.




The retry operates on a specific FAILED order. It acquires a per-state lock to prevent double-retries, replays the Pulpo completion with the label data already stored in the state's deferred writes, and transitions to COMPLETED. No active-states check is needed because the retry only touches this one order's Pulpo completion -- it does not interact with the machine or any other orders.

### Multi-Document Safety

When an order has been through the machine multiple times (e.g. ejected on machine A, then re-scanned on machine B), multiple state documents may exist for the same `packing_order_id`. The skip logic evaluates **all** documents:

| Documents Found | Classification | Webhook Decision |
|---|---|---|
| 1 EJECTED (machine A) + 1 INDUCTED (machine B) | 1 active, 1 terminal | **Skip** -- machine B is actively processing |
| 1 EJECTED (machine A) + 1 FAILED (machine B) | 0 active, 2 terminal | **Proceed** -- no machine is actively processing. Note: FAILED means machine B already labeled the package (see Section 4). Since deferred writes never ran, no tracking code exists in Pulpo, so the downstream idempotency check will **not** catch this — manually closing the box in Pulpo would generate a duplicate shipment. FAILED orders should be resolved via Dashboard retry, not manual packing. |
| 1 COMPLETED (machine A) + 1 COMPLETED (machine B) | 0 active, 2 terminal | **Proceed** -- all attempts finished |
| None | 0 active, 0 terminal | **Proceed** -- order never entered a machine |

The rule is simple: **if any document is in an active state, the webhook skips. If all are terminal or none exist, the webhook proceeds.**

### Machine Outage and Recovery

When the machine stops unexpectedly (power failure, mechanical jam, software crash), items may be physically stuck on the conveyor with their state documents still in active states. This section explains how the system recovers.

#### Why Active States Cannot Get Permanently Stuck

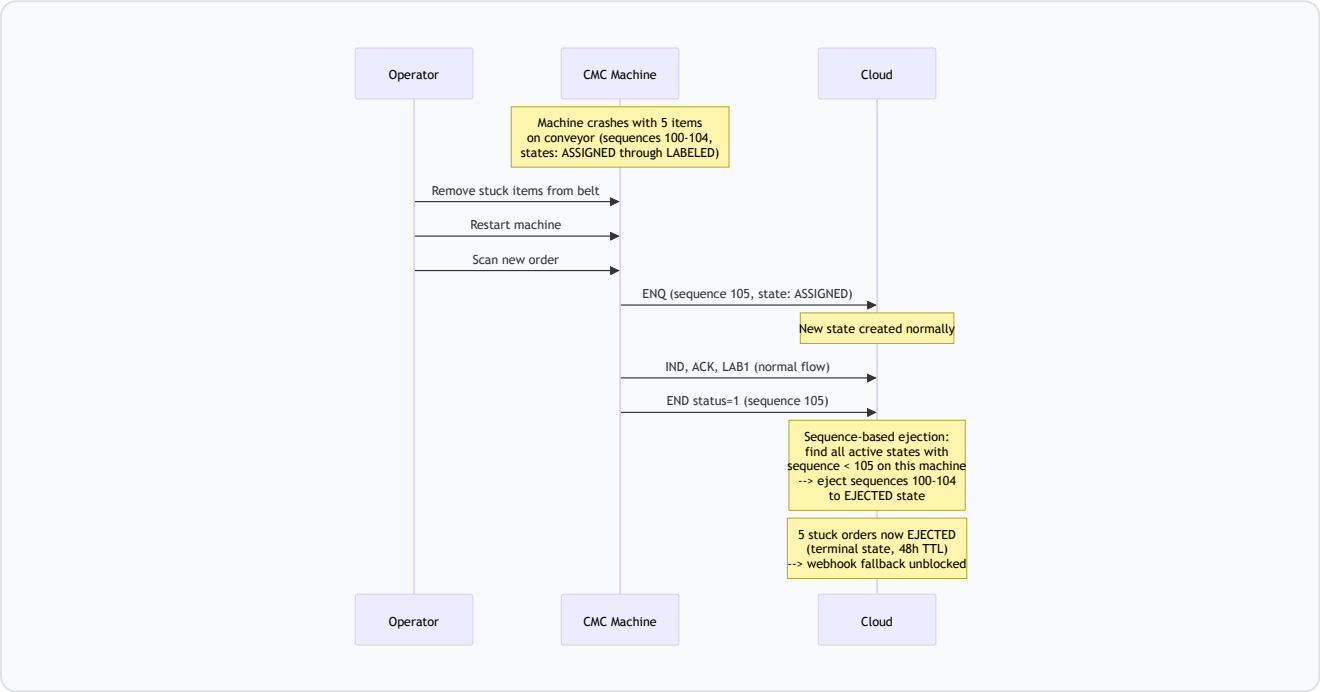
The CMC is a physical conveyor belt. Items move from station to station. If the machine stops mid-processing:

1. Items are **physically stuck on the belt**
2. An operator **must remove them** before restarting (otherwise the machine jams on restart)
3. The machine cannot "forget" about items -- they are physically present until removed

This means active states always correspond to a physical reality: either the item is on the conveyor (machine running), or the item was removed and the machine will clean up the state when it resumes.

### Self-Healing via Sequence-Based Ejection

The sequence-based ejection mechanism (described in detail in [Section 7: Automatic Recovery Mechanisms](#1-sequence-based-ejection-at-end)) ensures that stale active states are cleaned up automatically. When an order reaches the END station, the system ejects all older active states on that machine:




**One successful order through the restarted machine cleans up all prior stuck states automatically.**

### What Happens to the Stuck Orders During Downtime?

During the window between "machine goes down" and "machine processes one new order to END," the specific orders that were on the conveyor remain in active states. This affects only those orders:

| Order Category | Count (example) | Status During Downtime |
|---|---|---|
| Orders that were on the conveyor | However many were physically on the belt | Active state blocks webhook -- **cannot be packed manually until machine recovers or admin intervenes** |
| Orders picked to machine location but never scanned | 695 | No state document -- **can be packed manually immediately** |

The number of affected orders is inherently limited by how many items physically fit on the conveyor belt at once. Orders that never entered the machine have no state document and are completely unaffected.

### Recovery Options

| Situation | Recovery | Time to Resolve |
|---|---|---|
| Machine restarts and processes a new order | Automatic -- sequence-based ejection cleans up all stale states | Minutes (one order cycle) |
| Machine is down but will restart soon | Wait for restart, then see above | Until restart + one order |
| Machine is down for extended time, need to ship the stuck orders now | Admin deletes or resolves the states via CMC Dashboard | Immediate (manual action) |
| Machine is permanently decommissioned | Admin cleans up all states and machine config via Dashboard | One-time cleanup |

### Why No Automated Sweeper Is Needed

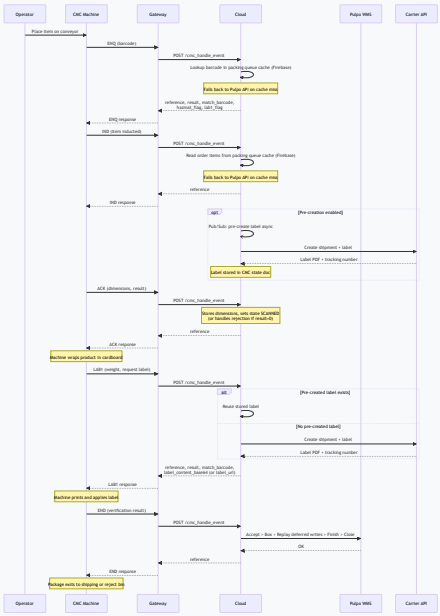
A scheduled background job to clean up stale active states was considered but is unnecessary:

- The number of affected orders during an outage is small (limited by physical conveyor capacity)
- The machine's sequence-based ejection self-heals within one order cycle after restart
- The CMC Dashboard provides immediate manual cleanup if needed
- Adding a time-based sweeper would introduce a magic threshold constant and risk prematurely ejecting states during legitimate slow processing or extended machine pauses

---

## Appendix: Communication Flow per Event

For reference, the full request/response data flow for each protocol event:




---

## Changelog

| Version | Date | Summary |
|---|---|---|
| 2.0 | 2026-03-10 | Removed REVIEW_REQUIRED state -- rejected/removed items always delete state and return order to Pulpo queue |
| 1.0 | 2026-03-02 | Initial documentation |